

Variables, operators, expressions

Variable types

char v_char; // 1 byte, unsigned integer

int v_int; // variable, minimum 16bits; ideally

float v_float; // real numbers, single precision (32 bits)

double v_double; // real numbers, double precision (64 bits)

Type modifiers

Only applied to integers and chars

- **signed**: range is $[-2^{\text{width}-1}+1, 2^{\text{width}-1}-1]$; negative values with 2's complement
- **unsigned**: range is $[0, 2^{\text{width}}-1]$

Applied to integers

- **short**: minimum 16 bits
- **long long**: minimum 64 bits

Applied to integers and double

- **long**: minimum 32 bits

Checking variable size

`sizeof` operator returns the size of a variable or type

```
void main(void){  
    printf("Char: %ld\n", sizeof(char));  
    printf("Short: %ld\n", sizeof(short));  
    printf("Int: %ld\n", sizeof(int));  
    printf("Long: %ld\n", sizeof(long));  
    printf("Long long: %ld\n", sizeof(long long));  
}
```

```
Char: 1  
Short: 2  
Int: 4  
Long: 8  
Long long: 8
```

`sizeof` works for complex variables and arrays as well (returns length of array):

```
int array[20];  
printf("Array size: %ld\n", sizeof(array));
```

80 bytes

Signed values

The sign is represented with the left-most bit (the most significant)

- If the bit is 0 the value is positive
- If the bit is 1 the value is negative

Values are stored in **2's complement** ($\text{abs}(\text{value}) + \text{representation} = 2^w$)

```
signed int cs;  
cs = -1;  
printf("Signed: decimal=%d; hex=%x\n", cs, cs);  
cs = 0x80000000;  
printf("Signed: decimal=%d; hex=%x\n", cs, cs);  
cs -= 1;  
printf("Signed: decimal=%d; hex=%x\n", cs, cs);
```

ffffffff (4 bytes, $2^{4*8}-1$)

-2^{4*8-1} , min(signed int)

$2^{4*8-1}-1$, max(signed int)

Unsigned values

Always go from 0 to $2^{\text{width}}-1$, where *width* is the number of bits.

- There is no sign bit

What happens if we assign a negative value to an unsigned one?

1 → 0x0001

0 → 0x0000

-1 → 0xffff. Since there is no sign bit, this is equal to $2^{\text{width}}-1$

Overflowing counters (wrap-around)

Does this code complete?

```
int sum = 0;
unsigned short i;

for (i=100; i>=0; i--){
    sum += i;
}
```

Overflowing counters (wrap-around)

Does this code complete?

```
int sum = 0;
unsigned short i;

for (i=100; i>=0; i--){
    sum += i;
}
```

No. i will wrap around back to 2^w-1 after it reaches 0, therefore the test $i \geq 0$ will always be true.

Overflowing counters (wrap-around)

Does this code complete?

```
int sum = 0;
unsigned short i;

for (i=100; i>=0; i--){
    sum += i;
}
```

This needs a **signed** counter

```
int sum = 0;
short i;

for (i=100; i>=0; i--){
    sum += i;
}
```

No. i will wrap around back to 2^w-1 after it reaches 0, therefore the test $i \geq 0$ will always be true.

Wrap-around counters

Always be conscious of the type and range of your variables (especially counters)

Make sure that the values in your program are within the range of your variables.

Declaring variables in C

Alphanumeric characters (letters, digits, and “_”)

Cannot start with a digit

<type modifier> <type> <**name**>;

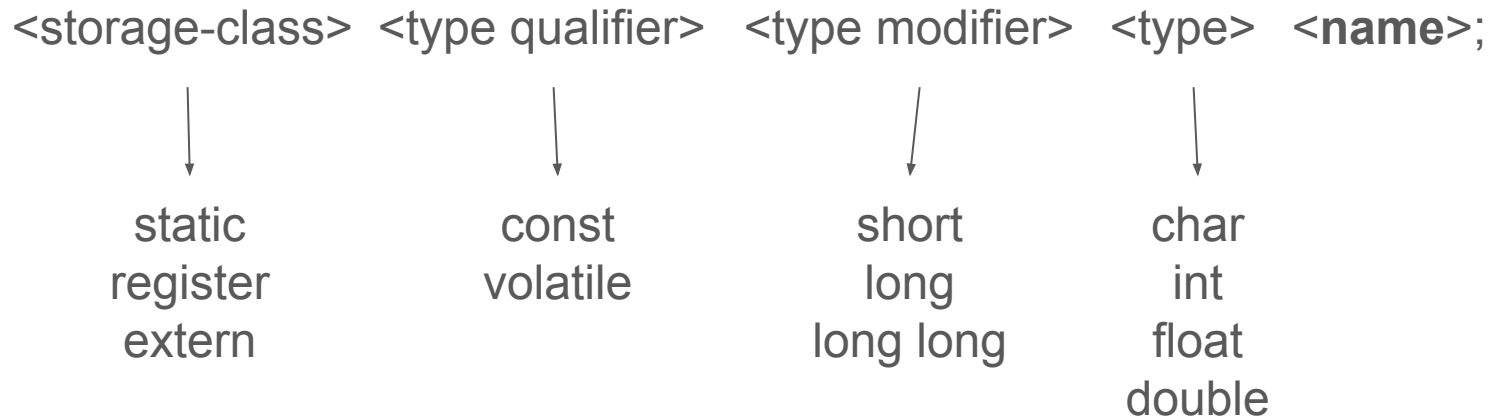
short
long
long long

char
int
float
double

Declaring variables in C

Alphanumeric characters (letters, digits, and “_”)

Cannot start with a digit



Type qualifiers

`const`

`volatile`

The **type qualifier** affects the way the compiler treats the variable storage area.

Const type qualifier

const marks a variable as unmodifiable.

Any modification of that variable is prohibited and will result in a compiler error.

Applied to basic types:

```
void main(void){  
    const int sum = 0;  
  
    sum++;  
}
```

```
gets_test.c: In function 'main':  
gets_test.c:6:8: error: increment of read-only variable 'sum'  
    sum++;  
    ^~
```

Const applied to arrays

When applied to arrays **const** affects *all the elements of the array*.

```
void main(void){  
    const int array[5];  
    array[0] = 5;  
}
```

```
gets_test.c: In function 'main':  
gets_test.c:5:14: error: assignment of read-only location 'array[0]'  
    array[0] = 5;  
            ^
```

Const applied to pointers

When applied to pointers **const** marks as unmodifiable the contents of the variable that is pointed to

- The address in the pointer can be modified
- The contents at the address cannot

This is not.

```
void main(void){  
    const int *p;  
    int v[1];  
    p = &v;  
    *p = 1;  
}
```

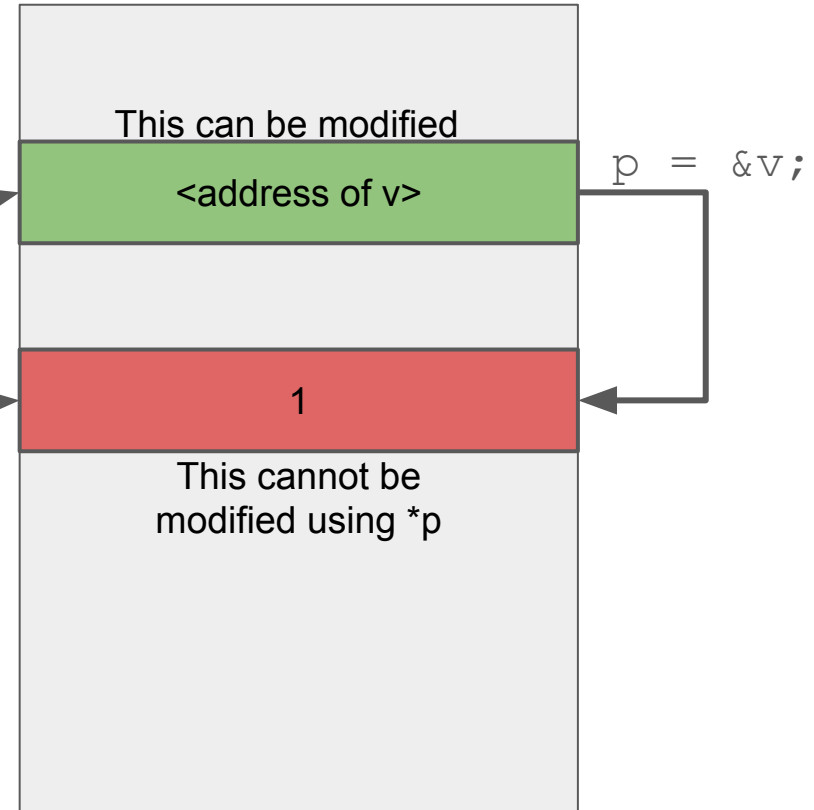
```
gets_test.c: In function 'main':  
gets_test.c:7:8: error: assignment of read-only location '*p'  
    *p = 1;  
    ^
```

This is fine.

```
void main(void){  
    const int *p;  
    int v = 1;  
    p = &v;  
}
```


Const applied to pointers

```
const int *p;  
int v;  
v = 1;  
p = &v;  
*p = 2;
```



Const function parameters

When used in a function parameter **const** represents a promise that the function will not change the variable contents. Helps with code readability.

- Only relevant to arrays and pointers - no use on value variables
- “The elements of the array will not change”
- “The variable pointed to will not change”

```
void increment(const int *source, int length){  
    int i;  
    for (i=0;i<length;i++){  
        source[i]++;  
    }  
}
```

test.c: In function 'increment':

```
test.c:7:18: error: increment of read-only location '*(source + (sizetype)((long unsigned int)i *  
4ul))'  
    source[i]++;  
               ^
```

Constant values

Numeric constants can be defined for integers and reals.

Character constants: single character between single quotes

- Example 'a', 'b', '1', '!'
- Non-printable characters: '\n', '\a', '\t'
- They are all converted to their ASCII integer representation

String constants

String constants: string of characters between double quotes

- Example “hello world”
- Can be split on multiple lines
- Consecutive string constants are automatically concatenated

```
"hello\  
world."  
"This is me.";
```

Internally the string is represented as an array of characters with an additional NULL ('\0') character at the end to mark the end of the string.

Array constants

One-dimensional array: {0, 1, 2, 3, 4, 5}

Two-dimensional array: {{0,1}, {2,3}, {4,5}}

- After all it is an array of arrays

Array of characters: {'h', 'e', 'l', 'l', 'o', '\0'}

- Without the NULL character at the end it does not work like a string

The **volatile** type qualifier

Not that common.

Used mainly when dealing with hardware or multi-threaded applications.

More information:

<https://barrgroup.com/embedded-systems/how-to/c-volatile-keyword>

Storage class specifiers

Register

Static

Extern

- They inform the compiler how (and where) a variable should be stored
- This then determines
 - When memory for the variable is allocated (static allocation)
 - How the variable is initialised
 - When the memory is freed.
- Related to the **scope** of variables

Variable scope

source1.c

```
int var1;  
int var2;
```

```
int func(char c){  
    Int var2;  
    Char var4;  
    ...  
}
```

```
void main(void){  
    Int var1;  
    Int var3;  
    ...  
}
```

source2.c

```
int var1;  
int var2;
```

```
int func(char c){  
    Int var2;  
    Char var4;  
    ...  
}
```

```
void main(void){  
    Int var1;  
    Int var3;  
    ...  
}
```


Variable scope

```
source1.c
-----
int var1;
int var2;

int func(char c){
    Int var2;
    ...
    {
        Char var4;
        ...
    }
}

void main(void){
    Int var1;
    Int var3;
    ...
}
```

Global scope

- * accessible throughout the entire file
- * not accessible outside the file

Local (block) scope

- * between { }
- * accessible only within the block ({ to })

```
source2.c
-----
int var1;
int var2;

int func(char c){
    Int var2;
    Char var4;
    ...
}

void main(void){
    Int var1;
    Int var3;
    ...
}
```

Variable scope: default behaviour

Without any storage class specifiers the following happens:

In the **global scope**

- Variables **explicitly** initialised (`int v = 13;`) are allocated memory in the **.data** section of the binary, and initialised with the specified value
- Variables **not explicitly** initialised (`int v;`) are allocated memory in the **.bss** section and initialised to zero
- Variables are available throughout the life of the binary

In the **local scope**

- Variables are allocated memory **on the stack**
- Variables are **not initialised** (initial value is unspecified)
- When the block completes (exits) the variable is no longer valid

Local scope variables

Local scope == function (in most cases).

When we exit a function, all the variables declared therein will be released from memory.

This means:

- The function can return **values** (scalars)
- The function **should not** return addresses to variables declared within the function.

Why? Because those addresses are no longer in use → unspecified behaviour.

Scope and accessibility

Local variables are only accessible within their code block.

Global variables are accessible everywhere, in all functions.

Local variables will shadow global variables with the same name.

- In general, you shouldn't do it.

Static local variables

Converts a local variable into a global variable

- The variable is allocated in the .bss or the .data sections
- The variable will keep its value across multiple function invocations.

```
#include <stdio.h>
void add(int val){
    int non_static = 0;
    static int static_var = 0;
    non_static = non_static + val;
    static_var = static_var + val;
    printf("Non static: %d; Static: %d\n",
           non_static, static_var);
}
void main(void){
    add(1); add(1); add(1);}
```

```
> ./test
Non static: 1; Static: 1
```

Static local variables

Converts a local variable into a global variable

- The variable is allocated in the .bss or the .data sections
- The variable will keep its value across multiple function invocations.

```
#include <stdio.h>
void add(int val){
    int non_static = 0;
    static int static_var = 0;
    non_static = non_static + val;
    static_var = static_var + val;
    printf("Non static: %d; Static: %d\n",
           non_static, static_var);
}
void main(void){
    add(1); add(1); add(1);}
```

```
> ./test
Non static: 1; Static: 1
Non static: 1; Static: 2
Non static: 1; Static: 3
```

Static local variables

Just because the variable is now global, doesn't mean that it can be accessed outside the local scope.

```
void test_static(int plus){
    static int c = 0;

    c += plus;
    printf("%d\n", c);
}

void main(void){

    test_static(1);
    test_static(1);
    test_static(1);
    test_static(1);
    printf("%d\n", c);
}
```

```
test.c:16:20: error: 'c' undeclared (first use in this function)
    printf("%d\n", c);
                   ^
```

Extern variables

- Multi-file (multi-module) application
- What if we want to use a variable defined in another module?
- Global variables are global at file scope, they cannot be accessed in another file

Solution

- Define a global variable in one module
- In the other module(s), declare the variable in the other module as “extern”

Extern variables

module1.c

int myvar;

```
int func(char c){  
...  
}
```

```
void main(void){  
...  
}
```

var_user_1.c

extern int myvar;

```
int func(char c){  
...  
}
```

```
void main(void){  
...  
}
```

var_user_2.c

extern int myvar;

```
int func(char c){  
...  
}
```

```
void main(void){  
...  
}
```

Extern variables

- Extern variables work as references (except for the memory management)

Are they useful?

- Error management in libc: **errno** global error code, can be set and read by any function in an application
- Linux Kernel
- Larger projects

Cast and truncation

Implicit conversion/truncation

C automatically converts and truncates types:

- In assignments, when a wider (larger) data type is assigned to a narrower one.
- In expressions containing different types

Implicit conversion/truncation

In assignments, wider types are truncated to narrower by dropping excess data

- Longer ints will lose the high bytes when assigned to shorter ints
- Floats are rounded when assigned to ints

```
unsigned short p = 98304;

printf("98304=0x%x\n", 98304);
printf("Size of unsigned short=%d\n", sizeof(unsigned short));
printf("p=%hu (0x%x)\n", p, 0);
```

Unsigned short is 2B.
Goes from
 $0 \rightarrow 2^{32}-1$ (65535)

```
gets_test.c:4:30: warning: large integer implicitly truncated to unsigned type [-Woverflow]
    const unsigned short p = 100000;
```

```
    const unsigned short p = 100000;
```

```
98304=0x18000
```

```
Size of unsigned short=2
```

Implicit conversion/truncation

In assignments, wider types are truncated to narrower by dropping excess data

- Longer ints will lose the high bytes when assigned to shorter ints
- Floats are rounded when assigned to ints

```
unsigned short p = 98304;

printf("98304=0x%x\n", 98304);
printf("Size of unsigned short=%d\n", sizeof(unsigned short));
printf("p=%hu (0x%x)\n", p, 0);
```

Unsigned short is 2B.
Goes from
 $0 \rightarrow 2^{32}-1$ (65535)

```
gets_test.c:4:30: warning: large integer implicitly truncated to unsigned type [-Woverflow]
    const unsigned short p = 100000;
```

```
    const unsigned short p = 100000;
```

```
98304=0x18000
Size of unsigned short=2
p=32768 (0x8000)
```

Implicit casting: expression rules

In expressions narrower values are up-cast to wider types:

- if either operand is **long double** convert the other to **long double**
- If either operand is **double** convert the other to **double**
- If either operand is **float** convert the other to **float**
- Convert **char** and **short** to **int**
- If either operand is **long** convert the other to **long**

Implicit casting in expressions: example

Integer vs float division.

```
float b;  
int c = 10;  
  
b = c/3;  
printf("%0.2f\n", b);  
b = c/3.1;  
printf("%0.2f\n", b);
```

```
> ./test
```


Implicit casting in expressions: example

Integer vs float division.

```
float b;  
int c = 10;  
  
b = c/3;  
printf("%0.2f\n", b);  
b = c/3.1;  
printf("%0.2f\n", b);
```

```
> ./test  
3.00  
3.23
```



Implicit casting

In assignments we lose data due to truncation.

In expressions the data is up-converted to the widest type. This preserves the data.

Explicit casting

The cast operator (same as in Java).

```
float b;  
int c = 10;  
  
b = c/3;  
printf("%0.2f\n", b);  
b = (float)c/3;  
printf("%0.2f\n", b);
```

```
> ./test  
3.00  
3.33
```

Explicit casting and pointers

By default pointers are of type `void *`

- This is because they point to a generic memory location
- **malloc** actually returns a `void *`
- It is up to the developer to cast the contents of the memory to the proper type.

Explicit casting of pointers

1. With const function parameters
2. With parameters taking `void *`

Fun with casting pointers

```
int value = 0x12345678;  
  
int *int_pointer = &value;  
short *short_pointer = (short*)&value;  
  
printf("0x%x\n", *int_pointer);  
printf("0x%hx\n", *short_pointer);
```

```
> ./test  
0x12345678
```

Fun with casting pointers

```
int value = 0x12345678;  
  
int *int_pointer = &value;  
short *short_pointer = (short*)&value;  
  
printf("0x%x\n", *int_pointer);  
printf("0x%hx\n", *short_pointer);
```

```
> ./test  
0x12345678  
0x5678
```

Fun with casting pointers

```
int v1=0x646c72;  
int v2 = 0x6f77206f;  
int v3 = 0x6c6c6568;  
  
printf("0x%x\n", v1);  
printf("0x%x\n", v2);  
printf("0x%x\n", v3);
```

```
$ ./test  
0x646c72  
0x6f77206f  
0x6c6c6568
```


Fun with casting pointers

```
int v1=0x646c72;  
int v2 = 0x6f77206f;  
int v3 = 0x6c6c6568;  
  
printf("0x%x\n", v1);  
printf("0x%x\n", v2);  
printf("0x%x\n", v3);  
printf("%s\n", (char*)&v3);
```

```
$ ./test  
0x646c72  
0x6f77206f  
0x6c6c6568
```

Fun with casting pointers

```
int v1=0x646c72;  
int v2 = 0x6f77206f;  
int v3 = 0x6c6c6568;  
  
printf("0x%x\n", v1);  
printf("0x%x\n", v2);  
printf("0x%x\n", v3);  
printf("%s\n", (char*)&v3);
```

```
$ ./test  
0x646c72  
0x6f77206f  
0x6c6c6568  
hello world
```



Operators

Precedence	Type	Operators	Associativity
1	Postfix	() [] -> . ++ --	Left to right
2	Unary	+ - ! ~ ++ -- (type) * & sizeof	Right to left
3	Multiplicative	* / %	Left to right
4	Additive	+ -	Left to right
5	Shift	<< >>	Left to right
6	Relational	< <= > >=	Left to right
7	Equality	== !=	Left to right
8	Bitwise AND	&	Left to right
9	Bitwise XOR	^	Left to right
10	Bitwise OR		Left to right
11	Logical AND	&&	Left to right
12	Logical OR		Left to right
13	Conditional	?:	Right to left
14	Assignment	= += -= *= /= %= >>= <<= &= ^= =	Rigth to left
15	Comma	,	Left to right

Precedence	Type	Operators	Associativity
1	Postfix	() [] -> . ++ --	Left to right
2	Unary	+ - ! ~ ++ -- (type) * & sizeof	Right to left
3	Multiplicative	* / %	Left to right
4	Additive	+ -	Left to right
5	Shift	<< >>	Left to right
6	Relational	< <= > >=	Left to right
7	Equality	== !=	Left to right
8	Bitwise AND	&	Left to right
9	Bitwise XOR	^	Left to right
10	Bitwise OR		Left to right
11	Logical AND	&&	Left to right
12	Logical OR		Left to right
13	Conditional	?:	Right to left
14	Assignment	= += -= *= /= %= >>= <<= &= ^= =	Rigth to left
15	Comma	,	Left to right

The increment/decrement operators

`j++;` `--j;` Nothing fancy, we have this in Java as well.

How do they work?

```
int a = 0;
int b = 0;

a = b++;
printf("a=%d b=%d\n", a, b);

a = ++b;
printf("a=%d b=%d\n", a, b);
```

```
> ./test
```

The increment/decrement operators

`j++;` `--j;` Nothing fancy, we have this in Java as well.

How do they work?

```
int a = 0;
int b = 0;

a = b++;
printf("a=%d b=%d\n", a, b);

a = ++b;
printf("a=%d b=%d\n", a, b);
```

```
> ./test
a=0 b=1
```

The increment/decrement operators

`j++; --j;` Nothing fancy, we have this in Java as well.

How do they work?

```
int a = 0;
int b = 0;

a = b++;
printf("a=%d b=%d\n", a, b);

a = ++b;
printf("a=%d b=%d\n", a, b);
```

```
> ./test
a=0 b=1
a=2 b=2
```


The increment/decrement operators

`j++`; `--j`; Nothing fancy, we have this in Java as well.

How do they work?

- Only in an expression (right hand side of assignment)
- `j++` (*postfix*) → use the value of `j` in the expression; increment `j`
- `++j` (*prefix*) → increment `j`; use its value in the expression.

The increment/decrement operators

Copying a source string into a destination string

```
while (source[i] != '\0') { dest[i] = source[i]; i = i+1; }
```

Another version using

- Postfix increment operator
- The fact that an assignment returns the assigned value (A=B returns B)

```
while ((dest[i] = source[i++]) != '\0');
```

Bitwise operations

Operations that modify the bits of a number:

- Shifting the number right ($\gg$) or left ($\ll$)
- AND ($\&$), OR ($\mid$), XOR ($\wedge$)

These also exist in Java and Python with the same meaning.

Bitwise shift

```
int i=1;  
printf("i=%d. In hex: 0x%x\n", i, i);  
i = i << 1;  
printf("i=%d. In hex: 0x%x\n", i, i);  
i = i << 1;  
printf("i=%d. In hex: 0x%x\n", i, i);
```

```
> ./test  
i=1. In binary: 0x001
```

Bitwise shift

```
int i=1;

printf("i=%d. In hex: 0x%x\n", i, i);
i = i << 1;
printf("i=%d. In hex: 0x%x\n", i, i);
i = i << 1;
printf("i=%d. In hex: 0x%x\n", i, i);
```

```
> ./test
i=1. In binary: 0x001
```

Bitwise shift

```
int i=1;

printf("i=%d. In hex: 0x%x\n", i, i);
i = i << 1;
printf("i=%d. In hex: 0x%x\n", i, i);
i = i << 1;
printf("i=%d. In hex: 0x%x\n", i, i);
```

```
> ./test
i=1. In binary: 0x001
i=2. In binary: 0x010
```

Bitwise shift

```
int i=1;

printf("i=%d. In hex: 0x%x\n", i, i);
i = i << 1;
printf("i=%d. In hex: 0x%x\n", i, i);
i = i << 1;
printf("i=%d. In hex: 0x%x\n", i, i);
```

```
> ./test
i=1. In binary: 0x001
i=2. In binary: 0x010
i=4. In binary: 0x100
```

Left shifting (<<) a number shifts all its bits to the left and adds a zero bit at the end

- The effect is multiplying the number by 2

Right shifting (>>) shifts the bits to the right.

- The effect is dividing the number by 2

Bitwise AND

Boolean AND operation: $0 \& 0 = 0$; $0 \& 1 = 0$; $1 \& 0 = 0$; $1 \& 1 = 1$.

```
int a = 5; int b = 6;  
printf("%d\n", a & b); → outputs 4;
```

- Performing a bitwise AND (&) between two numbers returns a number that has a bit of 1 where a and b **both** have a 1, and zero elsewhere.
- $a = 5 \rightarrow 101$ (in binary)
- $b = 6 \rightarrow 110$
- $a \& b \rightarrow 100$

Bitwise OR

Boolean OR operation: $0 \mid 0 = 0$; $0 \mid 1 = 1$; $1 \mid 0 = 1$; $1 \mid 1 = 1$.

```
int a = 5; int b = 6;
```

```
printf("%d\n", a | b); → outputs 7;
```

- Performing a bitwise OR(|) between two numbers returns a number that has a bit of 1 where either a or b have a 1, zero elsewhere
- $a = 5 \rightarrow 101$ (in binary)
- $b = 6 \rightarrow 110$
- $a \mid b \rightarrow 111$

Bitwise XOR (eXclusive OR)

Boolean XOR operation: $0 \wedge 0 = 0$; $0 \wedge 1 = 1$; $1 \wedge 0 = 1$; $1 \wedge 1 = 0$.

```
int a = 5; int b = 6;  
printf("%d\n", a ^ b); → outputs 3;
```

- Performing a bitwise XOR(^) between two numbers returns a number that has a bit of 0 where a and b have the same bit, 1 elsewhere
- $a = 5 \rightarrow 101$ (in binary)
- $b = 6 \rightarrow 110$
- $a \wedge b \rightarrow 011$

Bitwise negation ($\sim$)

Bitwise negation ($\sim$) inverts the bits of a number.

So 100 becomes 011.

What are they good for?

Checking the value of individual bits:

1. Build a mask for a certain bit position: `int mask = 1 << bitposition;`
2. Check if bit is set in number: `if (number & mask) ...`
3. Check if bit is cleared in number: `if (~number & mask) ...`

```
void to_binary(int number){  
    while (number > 0){  
        printf("%d", number & 1);  
        number >>= 1;  
    }  
    printf("\n");  
}
```

What are they good for?

Checking the value of individual bits:

1. Build a mask for a certain bit position: `int mask = 1 << bitposition;`
2. Check if bit is set in number: `if (number & mask) ...`
3. Check if bit is cleared in number: `if (~number & mask) ...`

```
void to_binary(int number){  
    while (number > 0){  
        printf("%d", number & 1);  
        number >>= 1;  
    }  
    printf("\n");  
}
```

```
void to_binary(int number){  
    if (~number == 0){  
        printf("0b");  
    }else{  
        to_binary(number >> 1);  
        printf("%d", number & 1);  
    }  
}
```

What are they good for?

Setting bits:

1. Build a mask for a certain bit position: `int mask = 1 << bitposition;`
2. Set the corresponding bit: `number = number | mask;`

Clearing bits: `number = number & ~mask;`

Toggling bits: `number = number ^ mask;`

Evaluating expressions

Expression evaluation

When we have complex expressions

```
if ((a=F(x)) || (a=a+1, b=G(a))) && H(a,b))
```

How are they evaluated and what is the final value of the expression?

Boolean operations (AND and OR)

It is important to know how they are evaluated.

With AND: `A && B && C && ... && Z`

- Evaluated from left to right until an expression is False

```

int f1(int val){
    printf("f1\n");
    return !(val % 2);
}

int f2(int val){
    printf("f2\n");
    return !(val % 3);
}

int f3(int val){
    printf("f3\n");
    return !(val % 5);
}

```

```

void main(){
    int i;

    i=10;

    printf("10:\n");
    if (f1(i) && f2(i) && f3(i)){
        printf("Yes\n");
    }

    i=12;
    printf("12:\n");
    if (f1(i) && f2(i) && f3(i)){
        printf("Yes\n");
    }

    i=14;
    printf("14:\n");
    if (f1(i) && f2(i) && f3(i)){
        printf("Yes\n");
    }
}

```

Output:

```

10:
f1
f2

12:
f1
f2
f3

14:
f1
f2

```

Boolean operations (AND and OR)

It is important to know how they are evaluated.

With OR: $A \parallel B \parallel C \parallel \dots \parallel Z$

- Evaluated left to right until the first expression that is True

Comma operator

Allows several expressions to be grouped together in a single statement:

```
int a = 1;  
(a+1) , (a++) , (a+1) ;
```

The value of a suite of expressions separated by commas is that of the right-most expressions. The above is processed as follows:

1. (a+1) evaluates to 2. Since there is a comma after, the value is discarded
2. a++ evaluates to a (1). a is incremented. The value of the expression is 1 but it is discarded because of the comma
3. (a+1) now evaluates to 3. No more commas so the combined expression evaluates to 3.

When is the comma operator used?

Declaring and initialising several variables of the same type, on the same line:

```
int i=0, a, b, c=2;
```

- Here the comma acts as a separator, nothing is evaluated.

When is the comma operator used?

For loops are very expressive.

What does this function do?

```
Void f(const int *source, int *dest, int length){  
    int i=0;  
    for (; i<length; dest[i] = src[i], i++);  
}
```

When is the comma operator used?

For loops are very expressive.

What does this function do?

```
Void f(const int *source, int *dest, int length){  
    int i=0;  
    for (; i<length; dest[i] = src[i], i++);  
}
```

The for loop is equivalent to

```
for (i=0; i<length; i++) dest[i] = src[i];
```

or

```
for (i=0; i<length; dest[i++]=src[i]);
```

Comma operator in for loops

Should only be used in the **initializer** and the **next step**, not in the condition.

If used in the condition it might not achieve the desired result because

- (A,B,C,...,Z) has the value Z
- A condition usually considers the conjunction (AND), or disjunction (OR), or some combination of the terms.

A && B && C is true if all are true

A, B, C considers only C. So it is true if C is true, regardless of the other ones.

Evaluating complex expressions

```
if ((a=F(x)) || (a=a+1, b=G(a))) && H(a,b))
```

Evaluating complex expressions

```
if ( ( (a=F(x)) || (a=a+1, b=G(a)) ) && H(a,b) )
```

Of the form A && B so we evaluate A to see if it is True, then B.

Evaluating complex expressions

```
if ( ( (a=F(x)) || (a=a+1, b=G(a)) ) && H(a,b) )
```

Of the form $A \parallel B$ so we evaluate A ($a=F(x)$) and if it is False also B ($a=a+1$, $b=G(a)$).

Evaluating complex expressions

```
if ( ( (a=F(x)) || (a=a+1, b=G(a)) ) && H(a,b) )
```

Of the form $A \parallel B$ so we evaluate A ($a=F(x)$) and if it is False also B ($a=a+1$, $b=G(a)$).

- $a=F(x)$ will return the value $F(x)$. If that is 0 the statement is False

Evaluating complex expressions

```
if ( ( (a=F(x)) || (a=a+1, b=G(a)) ) && H(a,b) )
```

Of the form $A \parallel B$ so we evaluate A ($a=F(x)$) and if it is False also B ($a=a+1$, $b=G(a)$).

- $a=F(x)$ will return the value $F(x)$. If that is 0 the statement is False
- Supposing that $F(x) = 0$ we evaluate $a=a+1$, $b=G(a)$

Evaluating complex expressions

```
if ( ( (a=F(x)) || (a=a+1, b=G(a)) ) && H(a,b) )
```

Of the form $A \parallel B$ so we evaluate A ($a=F(x)$) and if it is False also B ($a=a+1, b=G(a)$).

- $a=F(x)$ will return the value $F(x)$. If that is 0 the statement is False
- Supposing that $F(x) = 0$ we evaluate $a=a+1, b=G(a)$
- Comma separated, evaluated left to right, takes the last value
- So the first part has the value $b = G(F(x)+1)$

Evaluating complex expressions

```
if ( ( (a=F(x)) || (a=a+1, b=G(a))) && H(a,b) )
```



$b = G(F(x)+1),$
assuming $F(x) \neq 0$

$H(a,b)$ now is evaluated as $H(F(x), G(F(x)+1))$